Recursive self-improvement for long-running agents

The problem it solves, how it differs from giving an agent a memory, and what the measurements show so far

SenseLab Research

https://sense-lab.ai

Preprint v0.1, October 2026

Abstract

An agent deployed into an operation that runs for months acts on knowledge that goes stale. Policies change, dependencies get re-pinned, a workflow is rewritten, and nobody tells the agent. The common answers are to trust the model, to put the agent's own history in its prompt, or to retrieve past notes from a vector store. Each of these can tell the agent what it did before. None of them can tell it that what it did before has stopped working, write down the corrected rule, prove the correction on the cases it failed, release it to a small share of traffic, and take it back if it does harm. That loop is what we mean by recursive self-improvement: a system whose own outcomes change the knowledge it acts on, under an evidence gate, with the loop itself subject to the same evidence. This paper explains the problem, states what such an engine has to do, and reports what SenseLab's engine measured on a preregistered continual benchmark in which the correct action for a task class changes part way through, for half of that class only. On four held-out seeds, two agent models and two task families (1,600 episodes per system), the engine takes the new action first in 45 % of the episodes whose rule moved, against 25 % for the frontier model alone, 32–36 % for three pgvector variants and 40 % for a 24k-token context window, while holding unchanged work at 100 % and costing a third of the context window. With ten agents sharing one engine over 200 episodes, the moved half rises from 37 % to 75 %. The gate held every correction it could not grade, shipped four it could, and stopped one wrong correction at 88 % recurrence against 10 % before it reached the shared record. We also report what did not separate: the engine's replay and canary gates add nothing measurable to outcome learning inside a 100-episode cell, and the context window keeps an unchanged rule better than the engine does. The case for recursive self-improvement rests on what memory cannot do, and the evidence here is the first measurement of an engine doing it.

1 The problem

Most evaluations of agents measure one task at a time. A model reads a task, acts, and is scored. Deployment is different in one way that matters more than any other: the agent keeps going. A support agent handles tickets for a year. A CI agent repairs builds for every commit a team pushes. An operations agent runs the same procedure every night. Over that time the world the agent works in changes, and almost none of the changes arrive as instructions.

Three properties of this setting decide what kind of system can work in it.

- *Tasks do not repeat.* Every ticket, build failure and incident is a new instance. Nothing can be memorised verbatim; what carries over is the rule, "in this situation, take this action".
- *Rules change silently and partially.* A policy is rewritten for one tier of customer and kept for the rest. A dependency is pinned for one class of package and left alone for others. The agent's old rule is now right half of the time, which is the hardest case: a system that only demotes what failed has no signal, because the rule still succeeds often enough to look fine.

- *The cost of a wrong rule compounds.* An agent that acts on a stale rule fails on every later instance of that situation, and so does every other agent that reads the same rule. A fleet of ten agents sharing one record repeats one mistake ten times per round.

The failure that matters is a confident action on what used to be true, repeated until a person notices. The question for anyone building agents for long-running work is how the system itself can notice, correct and verify, without a person in every loop, and without making things worse when its correction is wrong.

2 Three ways to keep an agent current, and where each stops

The approaches in use today put the burden in three different places.

Trust the model. The agent runs on a frontier model with a fixed prompt. Everything it knows about the operation is what the model learned in pretraining plus what the prompt says. This is the baseline every deployment starts from. It has one strength: the model's default policy is usually the textbook one, and when the operation follows the textbook the agent is right. It has one weakness that the benchmark below makes concrete: when the operation's rule differs from the textbook, the model has no way to learn that. In our confirmatory run, the frontier model alone takes the correct post-change action first in 25 % of the episodes whose rule moved and in 27 % of the episodes in the changed classes whose rule did not move. In the `support` family its moved-half rate is 0 of 150. It never learns the house rule because it has nowhere to write it.

Put the record in the prompt. The agent is given its own history: every prior note and outcome from the operation, most recent first, as many as fit. The model then reasons over the record. This works well on one thing. When a rule is non-standard and has not changed, the record is full of consistent examples and the model follows them: 92 % on the kept half in our run, the best of any system by a wide margin. It fails on the thing this paper is about. When a rule has changed, the record contradicts itself, old successes against new failures, and the model has to work out from raw history which half is current. It does so in 40 % of moved-half episodes. The approach also has two structural limits. The record grows without bound and must be truncated (24k tokens here), so the oldest evidence is the first to go, and every episode pays to re-read everything: $0.130 per episode, three times the cost of the engine and nine times the cost of the model alone. And it does not transfer: what one agent has in its prompt is invisible to the next agent until it is copied there.

Retrieve from a vector store. The agent writes notes after each task, embeds them, and retrieves the nearest ones before the next task. This is what most "agent memory" systems are [4, 5], with variations: a cross-encoder reranker to sharpen retrieval, or a per-entry win/loss counter so the agent can see how often a note led to success. We ran all three. Plain pgvector retrieval takes the new action first in 32 % of moved-half episodes; with a reranker, 36 %; with the win/loss counter, 32 %. All three keep an unchanged non-standard rule at 58–66 %, well below the context window's 92 %. The reason is in what retrieval answers. A vector store answers "what did I write that looks like this?". It does not answer "is what I wrote still true?". A reranker makes the first answer sharper and leaves the second unasked. A win/loss counter is a step toward the second answer, and in our run it did not help on the moved half and cost 12 points on the unchanged classes (84 % against 96 % for plain retrieval). A counter on an entry says how often it worked. It does not say when, or what to do instead. Mem0 and Zep were not run here; an earlier grid covers them [2].

The common thread is that all three treat knowledge as something to store and recall. What they leave out is a process: something that watches whether recalled knowledge still works, writes the corrected rule when it does not, tests the correction, and releases it carefully. That process is the subject of the next section.

3 What recursive self-improvement means here

We use the term in a narrow, operational sense. A system is recursively self-improving when its own outcomes change the knowledge it acts on, the changes are themselves tested against outcomes before they take effect, and the machinery that makes the changes is subject to the same evidence as the knowledge it changes. The word "recursive" points at the last clause: the corrections apply to the thing that produces corrections, and the record the loop keeps is also what trains its next version.

SenseLab's engine implements this as four levels. Everything below is production code; the benchmark calls the public API and the engine's own scheduler does the rest.

Level 0: outcomes decide what the agent acts on. Before acting, an agent asks the engine for a briefing and a retrieval on its situation. As it works it records the actions it takes and any attempt that failed. When the task ends it commits the outcome. The sealed trace carries exactly what was read, and the outcome is attributed to it: an entry read by a run that then failed is charged; an entry read by a clean success is credited. Every entry carries an evidence status (untested, validated, contested, discredited), and retrieval returns, with each hit, how each candidate action has fared here on similar tasks and a recommendation: act, explore, or escalate. The "explore" recommendation is assigned per agent, so ten agents facing the same uncertainty try different things instead of the same one.

This level already differs from a vector store in kind. The store answers by similarity. Level 0 answers by evidence attached to the similar thing, and that evidence comes from outcomes the engine observed, including the agent's own failed attempts. A rule that was validated three times and then fails is flagged as a *regime shift*: the world moved under a rule that used to be right. The engine also scopes the shift to a *situation*, a canonical label the agent attaches to each session, so a rule that broke for the paid tier is not demoted for the free tier.

Level 1: the engine proposes, tests and ships corrections. A regime shift proposes a fix automatically. SenseLab's repair agent, a platform-side component of the engine, reads the failing runs, the entry and the loop's history on that entry, and drafts either a corrected entry or a conditional procedure (conditions, steps, a verify step) on a branch of the shared record. It attaches a manifest: the runs it predicts it repairs, the runs it must not break, and an expected prevention rate. Nothing on the branch is visible to agents yet.

The draft then has to pass three gates.

1. *Stand-in replay.* A stand-in model re-runs each manifest case with the branch as its memory and the tool calls answered from the recorded run. A replay that could not be answered from the record (fidelity below 0.5) does not count as a pass.
2. *The customer's own replay.* The case set is delivered to the customer's agent, which re-runs it on the branch. The fix must turn the failed runs into successes and must not break the passed ones. A success reached only after a recorded failed attempt fails the case.

3. *A live canary.* Sessions are bucketed by a hash of (fix, session); a share reads from the branch, the rest from the main record, and writes always go to main. The canary is the harm gate [3]: the replays prove the benefit on the runs that failed, and the canary has to show that the change does not make the agent worse on the live traffic the old rule still served. It passes when the canary arm recurs significantly less than control, or when a one-sided 95 % bound on the difference in recurrence is under a 10-point margin while the failure is still live in control. It fails when the canary arm recurs significantly more.

A fix that passes is merged into the main record. Its prevention is verified over the following window and a fix that regresses is rolled back to the pre-fix versions. A policy can instead ship on a passed replay, skipping the canary; that is the "− canary" ablation below. A set of continuous judges grades sealed traffic and opens incidents, the second source of proposals after regime shifts.

Level 2: the loop reads its own history. Every event in a fix's life (proposed, tested, replayed, shipped, regressed, rolled back) is written back as evidence on the entry it concerned, and the repair agent's next draft on that entry reads it. A loop that proposes the same wrong fix twice has not learned; this level is where it can.

Level 3: the record trains the next model and the next improver. The traces the loop produces, including the repair trajectories and the cases they pinned, are a training set. Smaller models are distilled from them and served with the same evidence record as their memory. The repair prompt itself is A/B tested across proposals and scored by proven fixes.

Why the gate is the point. A model that drafts fixes from failure evidence will draft wrong ones. In the exploratory run reported below, one draft said that a timed-out export should be split into date ranges; the rule had moved to a settings change, and the draft would have failed 88 % of the sessions it touched. A system that writes such a draft directly into what every agent reads damages every session at once. The discipline that makes self-improvement usable in production is the order of operations: evidence before proposal, proof before release, a small exposure before full exposure, reversibility after, and an audit trail throughout. Generating corrections is the easy part; the model does that. The engine's value is in refusing most of them, and in keeping a record of what it refused and why.

4 What the measurements show

The numbers in this section come from a preregistered study of the whole engine [1]. The hypotheses, arms, metrics, seeds and analysis were registered before the first run; two seeds were used for development and four held-out seeds ran once after the analysis code was frozen. This paper quotes the confirmatory read, and labels the two development-seed probes and the one exploratory run as such.

The benchmark. Two task families with tool-using agents and oracle-graded outcomes: `support` (a helpdesk agent with account, order and export tools; 15 issue classes) and `ci-fix` (an agent repairing failing CI on a repository; 10 failure classes). Every task instance is generated and never repeated, with random identifiers so nothing can be read off an id. Each cell is one agent identity working 100 episodes with one store seeded with 300 distractor entries. At episode 30 the correct action for a subset of classes changes *conditionally*: it moves for instances with one attribute value

Table 1: Confirmatory read, sixteen cells per system (4 seeds × 2 families × 2 models), 1,600 episodes per system. First-attempt success before the change and, after it, on the three populations; escalations are episodes the agent handed off; cost is the all-in dollar cost per episode, including for SenseLab the loop's judges, replays and drafts.

system	pre-change	moved half	kept half	unchanged	escalated	$/episode
Frontier alone	65 %	25 % (72/284)	27 % (75/276)	77 % (432/560)	238	0.014
Context window (24k)	86 %	40 % (115/284)	92 % (253/276)	97 % (542/560)	124	0.130
pgvector	82 %	32 % (91/284)	66 % (183/276)	96 % (539/560)	131	0.024
pgvector + rerank	77 %	36 % (101/284)	66 % (183/276)	91 % (511/560)	165	0.027
pgvector + outcomes (DIY)	76 %	32 % (90/284)	58 % (161/276)	84 % (470/560)	189	0.032
SenseLab	85 %	45 % (127/283)	69 % (191/276)	100 % (556/557)	80	0.042

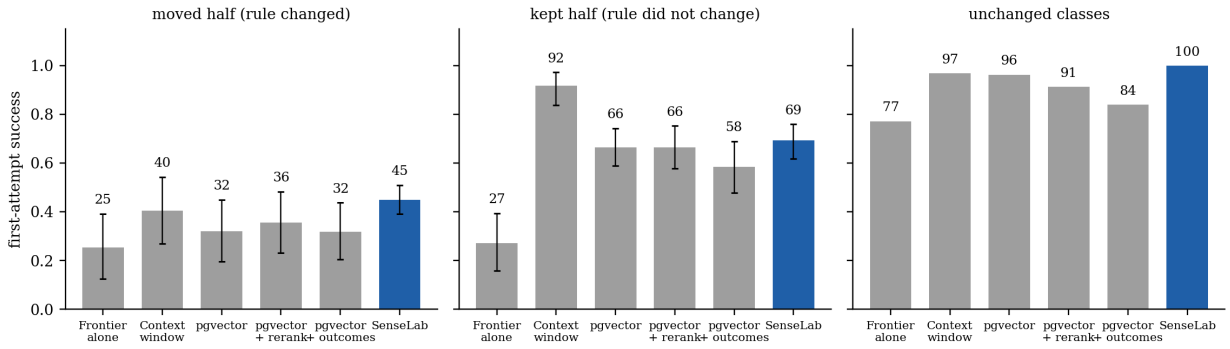


Figure 1: The three populations of Table 1. Bars are first-attempt success after the change; whiskers on the moved and kept halves are cell-level bootstrap intervals over the sixteen cells.

(the *moved half*) and stays for the others (the *kept half*). Half of the post-change tasks come from the changed classes. Classes that did not change are the third population (*unchanged*). The design was chosen because on an unconditional change outcome learning alone recovers nearly everything; the conditional change is the case where demotion has no signal, because the old rule is still right half of the time.

Six systems ran on identical prompts and the same two agent models (GPT-5.5 and Claude Opus 5): the frontier model alone; the context window (every prior note and outcome, most recent first, 24k tokens); pgvector (bge-small embeddings, cosine top-5 over notes); pgvector with the cross-encoder reranker SenseLab serves; pgvector with notes, periodic LLM consolidation and a per-entry win/loss counter shown to the agent (the "DIY outcomes" arm); and SenseLab, the whole engine with corrections shipped only after a passed canary. Four seeds, two families and two models give sixteen cells per system, 1,600 episodes each. The primary endpoint is first-attempt success on the moved half after the change; the secondary endpoints are first-attempt success on the kept half and on the unchanged classes.

4.1 After the rule changes

Table 1 and Figure 1 carry the main result. Before the change every system with a store is at 76–86 % and the model alone at 65 %: a store of any kind teaches the house rules. After the change the systems separate on the population the change created. On the moved half SenseLab takes the new action first in 45 % of episodes (127/283, cell-level interval [39, 51]); the context window is at

40 %, the pgvector arms at 32–36 %, the model alone at 25 %. On the unchanged classes SenseLab is at 100 % (556/557) and no system is above it. On the kept half the context window is ahead (92 % against 69 %), and the engine is level with the pgvector arms.

Paired on (family, seed, model) over the sixteen cells, SenseLab's moved-half rate is above every other system: +19 points over the model alone, +4 over the context window, +13 over pgvector, +9 over pgvector with the reranker and +13 over the DIY outcome store, winning 8 to 11 of the 16 cells in each pairing. The exact sign-flip test gives $p = 0.024$ against the model alone, 0.040 against pgvector and the DIY store, 0.119 against the reranker and 0.544 against the context window; after Holm correction across the five comparisons none is below 0.05 (0.121, 0.544, 0.159, 0.239, 0.159). The sign is right in every pairing and the sizes match the development seeds; sixteen cells is the power this study bought. The reader should take the deltas as estimates with their intervals, and the context-window comparison on the moved half as undecided.

The secondary endpoints are cleaner. On the unchanged classes SenseLab is above the model alone (+23, Holm 0.002), the reranker (+9, 0.035) and the DIY store (+16, 0.004), and indistinguishable from the context window and plain pgvector. On the kept half it is above the model alone by 38 points (14 of 16 cells, Holm 0.007), level with the three pgvector arms, and 24 points below the context window (1 of 16 cells, Holm 0.001). That last number is a real cost of the approach and we return to it in §6.

Where the gain comes from. The moved-half result is a `support` result. There SenseLab is at 42 % (62/149) against 0–27 % for every other system, with the model alone at 0 of 150: on house rules that differ from the textbook and then move, nothing without an outcome record learns the new rule at all. On `ci-fix` SenseLab is at 49 % (65/134) against 43–67 %, behind the context window (67 %). The two families differ in what the textbook gets right: in `ci-fix` the frontier model alone is at 54 % on the moved half, which suggests the post-change action is close to its default, and at 5 % (7/146) on the kept half, where a house rule it never learned still applies. By model, SenseLab is level with the context window on GPT-5.5 (36 % both) and ahead on Claude Opus 5 (54 % against 45 %); the pgvector arms swap order between the models (13–27 % on GPT-5.5, 37–51 % on Opus). The engine's effect does not depend on which model reads it.

4.2 The long horizon: ten agents, 200 episodes

A 100-episode cell with one agent is short for a loop whose gates need graded live traffic. The second protocol runs ten agents against one shared engine for 200 episodes, with the change at episode 50, on the same four held-out seeds, both families, GPT-5.5 (eight cells, 1,600 episodes). It is reported descriptively, as registered.

Figure 2 shows the curve. Pooled over the eight cells, moved-half first-attempt success is 37 % (40/109) in episodes 50–99, 55 % (60/110) in 100–149 and 75 % (67/89) in 150–199, with 88 % (15/17) in the last block. The kept half is 66, 76 and 72 % over the same windows. The unchanged classes are at 100 % in every post-change block (594/595). A rule that moved for half of a class at episode 50 is taken correctly three quarters of the time by episode 150, by ten agents each of which handled about a tenth of the episodes. What each inherits from the others is the outcome record and whatever the loop wrote.

4.3 What the loop did, and what it refused

The engine's own record is the second half of the evidence, and it is where the gate shows.

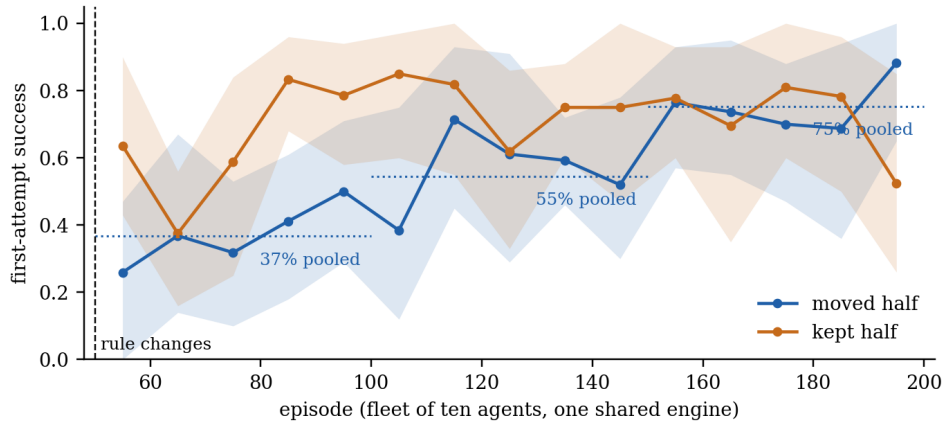


Figure 2: Fleet of ten agents sharing one engine, 200 episodes, change at episode 50; eight confirmatory cells pooled. First-attempt success per ten-episode block on the moved and kept halves, with cell-level bootstrap bands; dotted lines are the pooled moved-half rate over each fifty-episode window.

In the sixteen single-agent cells the loop proposed 32 corrections (23 from a regime shift, 9 from the judges). Sixteen passed the stand-in replay, six passed the customer's replay, five opened a canary, two shipped and none was reverted. The same loop with the canary removed, so that a passed replay ships, shipped 10 of 44. The canary held back four fixes in five that the replay would have released, in cells where no canary could reach a graded decision before the cell ended. In the eight ten-agent cells the loop proposed 48, the customer's replay passed 6 and failed 38, six canaries opened in six different cells, and none shipped: four reached the graded floor on both arms, one failed on a zero-count bound at 10 of 10 graded sessions, and three closed with both arms above the floor and no decision, because at 14 to 20 graded runs per arm the non-inferiority test cannot close a 10-point margin. The canary arm read its fix in 57 of 76 graded sessions (75 %) and the control arm in 0 of 54, so the gate was grading a treated arm against an untreated one, as designed.

The exploratory run sized the traffic to the gate. On the two development seeds, four cells of 400 episodes with every post-change episode drawn from the changed classes, the loop proposed 68 corrections; the stand-in replay passed 20, the customer's replay passed 8 and failed 22; eight canaries opened and five decided inside their cells. Four passed and shipped, three of them on a `ci-fix` class the fleet already handled (first-attempt success 0.89–0.97 before the ships, 0.89–0.96 after), so what the gate certified there is that the correction did no harm. One failed. The draft "csv export timeout → split date ranges" was the wrong action under the changed rule, which had moved to a settings change. Its canary arm graded 8 sessions, read the fix in 7, and recurred in 7 (88 %) against 2 of 20 in control (10 %; $p < 0.001$). The loop wrote the failure as a lesson to the main record within a second, the branch never reached main, and the next draft the repair agent produced for that class named the settings action. Nobody injected this case. It came out of the loop's own proposals, and the gate caught it.

What did not separate. Inside a 100-episode cell with one agent, the engine without its canary and the engine without its repair loop are indistinguishable from the whole engine: 44 / 71 / 99 % and 43 / 68 / 99 % (moved / kept / unchanged) against 45 / 69 / 100. Level 0, the outcome record, does most of the measurable work at this horizon. The repair loop's contribution is visible in its lifecycle counts and in the exploratory canary, and in the shape of the ten-agent curve; it is not visible in the single-agent aggregate. We say so because the case for the loop has to rest on what

it prevents as much as on what it adds, and the former is only measurable where it has traffic.

4.4 Two development-seed probes

Two probes from the development seeds bear directly on why a shared, outcome-attributed record matters for a fleet, and on level 3. Both ran on an earlier, unconditional change set and are development results.

A new agent joins. Six agents share one store. In one protocol one agent works alone for 60 episodes and five join; in the other, five work and a sixth joins at 60. A joining agent on SenseLab takes the post-change action on its first exposure to a changed class in 36 of 36 and 10 of 10 cases, with no stale picks. A joining agent on pgvector with a reranker does so in 31 % and 40 % of cases, with 17 % and 0 % stale picks and 16–20 % escalations. The same notes are in both stores. What the joiner inherits from the engine is the outcome record: which of those notes still works.

A small model trained on the record. Two small models were distilled from the development-seed `support` traces (200 traces each) and evaluated on three further held-out seeds (1,500 episodes). Alone, neither takes the new action first (9 % and 0 % on the moved half). Served with the engine's outcome record as their memory, both do (33 % and 30 %), at 1/23 of the frontier model's cost per episode, and ahead of GPT-5.5 with the same memory (22 %) at this rung. This is the first rung of an evaluation ladder and it uses the training classes; nothing is claimed from it beyond the direction. The point for this paper is that the record is model-independent: it was produced by one model's outcomes and lifted another.

5 What this means for building agents

The measurements support five statements about how to build agents for long-running work.

The unit of improvement is the operating rule. What goes stale is a rule of the form "in this situation, take this action". Retraining the model for every policy change is too slow and too coarse; editing the prompt by hand does not scale past a few agents. The rule is the thing to store, attribute outcomes to, correct, and test. Everything in Table 1 that separates the systems is about how each one handles that unit.

Evidence attached to knowledge beats more knowledge. The context window has strictly more information than the engine: it sees everything. It loses on the moved half because information without attribution is a contradiction the model has to resolve on every call. The engine's record is smaller and carries, with each rule, how it has fared since. On the kept half, where there is no contradiction, more information wins, and this is the engine's clearest weakness in the data.

The gate matters more than the generator. The repair agent drafted 68 corrections in the exploratory run and 32 and 48 in the confirmatory protocols. The customer's replay rejected most of them. The canary rejected one that the replay had passed and that would have harmed nine sessions in ten. A system whose corrections are gated by outcomes it did not write itself can be left running. One that ships on a model's say-so has to be watched.

A fleet learns once. A new agent on the engine took the new action on its first exposure in every case measured. The curve in Figure 2 is ten agents reaching 75 % on a rule most of them never saw fail. The cost of learning a changed rule is paid once per operation, and the record that pays it is the same record that trains the next model.

Self-diagnosis is an output. The study that produced these numbers found fourteen things about the engine's own evidence path, twelve of them defects, during the runs: a regime shift firing on entries that had never been validated, a replay grading on an outcome the agent reached only after a failed attempt, a canary arm that was not reading the fix it was supposed to be grading. Each was found because the loop keeps a record that can be checked against ground truth, each was fixed in production between runs, and each was re-measured on the same protocol [1]. A retrieval benchmark would have seen none of them, because none of them is about retrieval. A system that improves itself has to be inspectable in exactly this way, and the record it keeps for its own gates is what makes it so.

6 Limits

The evidence has limits, and some of them are findings about the approach.

The moved-half advantage over the context window is 4 points and does not approach significance ($p = 0.544$ paired). What separates the two systems is cost (3×) and what each can do for a second agent; this number does not. The kept-half loss to the context window (24 points, Holm 0.001) is the price of a selective, outcome-weighted record on a rule that did not move: the engine revises what the context window replays. Whether that trade is right depends on how often an operation's rules change, which is an empirical question about the operation.

Inside a 100-episode single-agent cell, the repair loop and the canary add nothing measurable to outcome learning. The loop's value is in the ten-agent curve, in what it refused, and in the one wrong correction it stopped; the single-agent aggregate does not show it, and we do not claim it does. In the eight confirmatory ten-agent cells nothing shipped at all, because the canary's rule needs more graded traffic per arm than 200 episodes across ten agents supplied. The four ships and the one failure come from an exploratory run on development seeds with the traffic concentrated on the changed classes, and three of the four ships were on a class that was already at ceiling.

The benchmark is two families of synthetic tasks with oracle grading and a rule change the authors wrote. It knows the truth before and after every change, which is what makes the measurement possible and what makes it an approximation of an operation whose truth nobody writes down. Horizons are 100 and 200 episodes. The engine's cost per episode is three times the model alone and 1.3–1.8 times a pgvector store, and a third of the context window.

7 Conclusion

An agent that runs for a long time needs more than a place to put what it learned. It needs a process that notices when what it learned has stopped being true, writes the correction, proves it on the cases that failed, exposes it to a little live traffic, and keeps or reverts it on the evidence. That process is recursive self-improvement in the sense this paper uses, and the measurements here are the first of an engine doing it in production code against baselines that only store and recall. The engine is ahead on the population the change created, level or ahead on what did not change, three times cheaper than the strongest baseline, and able to hand what it learned to a new agent or

a smaller model intact. It also records what it refused to ship, which is the part of a self-improving system that lets you trust the rest.

References

[1] SenseLab Research. Recursive self-improvement, gated by evidence: an engine that lets deployed agents propose, test, canary and ship corrections to their own operating knowledge, measured where the world changes under them. Preprint v0.3, October 2026. Companion paper; preregistration `HYPOTHESIS_RSI.md`, analysis tag `analysis-freeze-rsi`.

[2] SenseLab Research. From memory to recursive self-improvement: agents that learn from outcomes, agents that only remember, and a model trained on what they learned, when the tasks never repeat and the rules change. Preprint, 2026.

[3] R. Kohavi, D. Tang, Y. Xu. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, 2020.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023.

[5] P. Chhikara et al. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv:2504.19413, 2025.